Document 07 — AI Agents → Project 2

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

06_tools_function_calling

How to Read & Practice This Document

- **What:** the loop that turns one tool call into behavior with many steps, deciding its own next move.
- **Why:** "agent" stops being a buzzword once you've built this loop yourself, by hand, before ever using a library that does it for you.
- **When:** tasks that need several steps, decided one after another based on what the last step actually returned — not simple one-shot Q&A, which needs no loop at all.
- **How to practice:** 1. Read the material once, especially the ReAct paper's example. Just get the shape of it. 2. Trace a loop by hand on paper (**Basic**) before writing any code — this step is not optional. 3. Build the loop yourself (**Intermediate**) before using `create_agent` — you need to build this basic piece once, yourself. 4. Try **Project 2** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1 or Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud why every agent loop needs a hard limit on steps. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Doc06 gave the model exactly one shot per turn — call a tool, get a result, done, conversation over. Real tasks are rarely that tidy: they take several steps, and which step comes next depends on what the last one actually returned. This document is where Doc06's single round becomes a loop that keeps deciding its own next step, on its own.

First, the ReAct loop is Doc06's five-step round trip, repeated: register the task and tools, the model asks for one, your code runs it, you hand the result back, you ask again — nothing new gets invented here, the "reasoning" is just the model seeing each step's real result before choosing the next one. **Second**, every request and result gets appended to a growing message list, the loop's

own scratchpad — the model's next decision is only as good as what actually survives in that list, so a dropped or badly-written result makes the model effectively forget it happened. **Third**, the loop needs a hard stop that lives in *your* code, not the model's judgment, the same "a limit isn't optional" discipline Doc02 already taught you for retries. **Fourth**, once a loop runs many steps instead of one, the same handful of failures show up again and again: believable-but-wrong tool arguments, a tool error the model quietly ignores, and the infinite loop the step limit exists to prevent.

That's the whole story: one repeated round, a scratchpad to remember it by, and a limit to stop it. The Build Task wires your Doc06 tool library into exactly this loop, because Project 2 *is* that loop doing real, multi-step work with tools you already built.

Core Concepts (read this first — everything you need is here)

Topics on this page: The ReAct loop · The scratchpad · Setting a limit on the number of steps · Common single-agent failures · Agent memory · Planning before acting · When NOT to use an agent · Evaluating an agent

The ReAct loop: think → act → observe

An agent, with the fancy language stripped away, is Doc06's tool round-trip wrapped in a `while` loop and run more than once. Picture a shopkeeper who cannot see the store room from the counter: a customer asks for sugar and the cheapest oil, so he sends the shop boy to check stock, reads the answer, and *only then* decides the next errand — check the oil price. Nobody planned both errands up front; each was decided after the last one came back. The model is the shopkeeper, deciding one step at a time; your code is the shop boy, the only thing that actually does anything. "ReAct" is short for **Rea**son + **Act**: think, act, observe the real result, think again — until the model has enough to answer in plain text instead of asking for another tool.

How it really works

What/Why/When, named directly:

- **What:** Doc06's tool round trip (register → model asks → you run it → you hand the result back → ask again) wrapped in a loop that keeps repeating until the model's reply carries no more tool calls.
- **Why:** a single round trip only works when one tool call is the whole job. The moment step two depends on what step one's *real* result actually was, the model needs to see that result before it can decide the real next step — there is no way to write that decision down in advance.
- **When:** any task where the number and order of steps can't be known before the first tool result comes back. If you already know the fixed order today, skip the loop entirely — see When NOT to use an agent.

Two real ways to get this loop running, and why you build one before reaching for the other:

- **A hand-built loop, written by you. What:** the plain `while` / `for` loop below — call the model, check for tool calls, run them, append the results, ask again. **Why:** it's the only way to actually see what "a step," "a tool call," and "a final answer" are, in code you wrote and can read start to finish — every later library shortcut in this curriculum (`create_agent` , Doc09's graphs) is this exact loop with the wiring hidden, and hidden wiring only makes sense once you've seen it unhidden. **When:** always, at least once, before touching `create_agent` — this document's own Intermediate exercise requires it, and the Build Task only lets you pick a shape *after* you've built this by hand.
- **`create_agent` (LangChain 1.0+). What:** a library function that compiles the identical loop into a small LangGraph graph and runs it for you — one line instead of a `while` loop. **Why:** far less code for the same behavior, once you already know what that code is doing underneath — Doc09's own "tool-calling inside a graph" topic draws the graph this compiles to. **When:** real work, once the hand-built version has actually been built once — this document's Real-world exercise runs both side by side on the same prompts specifically so the diff teaches you something.

```
# create_agent — the same loop as the hand-built one below, library-shortened
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-4o-mini",
    tools=[get_weather, get_forecast],          # Doc06-style @tool functions
    system_prompt=SYSTEM_PROMPT,
)
task = "Should I bring an umbrella in Lahore tomorrow?"
result = agent.invoke(
    {"messages": [{"role": "user", "content": task}]},
    config={"recursion_limit": 14},   # ~2 graph steps per ReAct turn
)
```

- This is Doc06's "Seeing the whole thing three ways" round trip, repeated: register → model asks → you execute → you hand back → ask again. Doc06's chaining topic names this exact pattern — "adding the `while` makes it Doc07's agent, which names this pattern ReAct." Nothing new is invented here.
- The model API is stateless. Every turn re-sends the **whole** `messages` list from the start — the same fact as Doc06's round trip, just repeated N times instead of once.
- The model replies one of two shapes: plain text (`content` filled, `tool_calls` empty) — the final answer, stop the loop — or a tool request (`tool_calls` a list, `content` usually `None`) — run it and continue.
- Append order is fixed: the model's own tool-call message goes into `messages` **first**, then one `{"role": "tool", ...}` message per call. Reversing this, or skipping the first append, is Doc06's most common mistake, now inside a loop that repeats it every turn.

- Loop over **every** `message.tool_calls`, never just `[0]` — Doc06's parallel-tool-calls rule, now paid on every single turn of the loop instead of once.
- A tool failing mid-loop is not a special case: catch it and convert it to a clear string result exactly as Doc06's "Getting a tool's failure back to the model" topic requires, using Doc01's named-exception discipline to catch something specific rather than a bare `except:`.
- Use `temperature=0` for agents. A "creative" agent picks a different tool on different runs for the identical task, and an unrepeatable path is a path you cannot debug.
- Cost does not grow linearly with steps — each turn re-sends a longer history, so an 8-step run costs roughly the *sum* of a growing context, not 8× one call. This is Doc03's cost topic multiplied by every step of the loop.
- `recursion_limit` is `create_agent`'s hard stop, counting roughly **two** graph steps per ReAct turn — the mechanism itself belongs to Doc09, not repeated here; the counting trap is covered in full in Setting a limit on the number of steps below.
- Older tutorials use `AgentExecutor` or `create_tool_calling_agent` — both are legacy on LangChain 1.0+. `from langchain.agents import create_agent` is the current constructor; everything else about tool calling is unchanged.
- **A concrete trace, on Doc06's own weather tools:** task = *"Should I bring an umbrella in Lahore tomorrow?"* Step 1: the model has no `content`, one `tool_calls` entry — `get_forecast(city="Lahore", day="tomorrow")`. Your code runs it, gets back `"60% chance of rain"`, appends it as a `tool` message. Step 2: the model now has everything it needs — `content` = *"Yes, bring an umbrella — 60% chance of rain in Lahore tomorrow."*, `tool_calls` empty. The loop sees the empty list and returns. Nothing about this trace is special-cased; it's the general loop above, run twice.
- **Applied to Project 2's Worker:** this loop, unchanged, *is* the Worker's whole engine — Project 2's Step 2 has you build exactly the code above around one tool, and Step 3 grows it to 3+ tools with the limit and failure-recovery the next two topics cover.

Situation	What to do	Why
Next step depends on the last result	ReAct loop	Only the model, after seeing the real result, can pick the next move
Steps are always the same, same order	Plain function, no loop	Cheaper, faster, fully predictable — see When NOT to use an agent
One tool, one call, then answer	Doc06's single round	A loop that always turns once is just extra code
Steps unknown but must be reviewed first	Plan-first (see Planning before acting)	A human reads the plan before anything runs
Several specialists needed	Several agents (Doc11)	One agent with 15 tools chooses badly; three agents with 4 tools choose well

```
# the ReAct loop — Doc06's round trip, repeated. Logger is Doc01's.
from langchain_openai import ChatOpenAI

from exceptions import MaxIterationsExceeded
from logging_setup import get_logger
from tools import ALL_TOOLS


def run_agent(task: str, max_iterations: int = 6) -> str:
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [{"role": "user", "content": task}]
    steps = []
    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)     # THINK
        messages.append(response)            # the request goes in FIRST
        if not response.tool_calls:
            return response.content          # THINK said: done
        for call in response.tool_calls:     # ACT — every call, not one
            # run_tool(): find the tool, .invoke() it, return (result,
            # error) — you write it in the Intermediate exercise, which
            # also shows this same loop on the raw OpenAI SDK
            result, error = run_tool(call, ALL_TOOLS)
            if error is None:
                observation = result
            else:
                observation = f"Error: {error}"
            messages.append({                # OBSERVE
                "role": "tool",
                "content": observation,
                "tool_call_id": call["id"],
            })
            steps.append({"step": step, "tool": call["name"], "error": error})
        logger.info(f"step {step}: {len(response.tool_calls)} tool call(s)")
    message = f"no final answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

Common mistakes:

- *Mistake:* appending only the `tool` result, forgetting the model's own tool-call reply first. → *Symptom:* a `400` error about an unmatched `tool_call_id`, or the model repeats the same call forever. → *Fix:* `messages.append(msg)` first, always, then one `tool` message per call.
- *Mistake:* copying an old tutorial's `AgentExecutor` or `create_tool_calling_agent`. → *Symptom:* `ImportError`, or a deprecation warning, on LangChain 1.0+. → *Fix:* `from langchain.agents import create_agent` — everything else about tool calling still applies.

Where you'll meet it: this loop is the Build Task of this document and the heart of Project 2's Worker. Doc06's round-trip and failure-handling topics are exactly what runs inside each turn. Doc09 rebuilds the same behavior as an explicit graph you can pause and resume. Doc10 wraps it in routing and reflection patterns; Doc11 runs several of these loops side by side; Doc13 traces each turn; Doc14 breaks it on purpose.

Quick cheat sheet:

- An agent = a `while` loop around Doc06's round trip: model call → tool call → append result → model call again.
- The model only *asks*; your code is the only thing that *does*.
- Stop when `message.tool_calls` is empty, or when your own step counter runs out — whichever comes first.
- Append the model's reply **before** the tool result, and loop over **all** `tool_calls`, every turn.
- Build it by hand once (Project 2's Worker); use `create_agent` once you know what it's hiding — `recursion_limit` is its hard stop (Doc09 explains the graph underneath).

The scratchpad: how the loop remembers its own steps

As the loop runs, every request and result gets added to the growing `messages` list, sometimes called the scratchpad — this is Doc03's "no memory inside the model" fact, now happening *within* one run instead of across a whole chat. Picture the shopkeeper from the last topic with an unusual illness: he forgets everything the instant he finishes a sentence, and can only work from a notebook on the counter that someone reads to him in full before every decision. That notebook is the scratchpad. It is not a feature — it is the only reason the loop works. If a page is missing, that event never happened as far as the model knows; if a page is written badly, the model acts on the bad words, not on what really happened.

How it really works

What/Why/When, named directly:

- **What:** the growing `messages` list one run keeps — every request and every result, in order — re-sent in full on every model call.
- **Why:** the model has no memory of its own (Doc03's no-memory fact). Without this list re-sent every turn, the model would not know it already asked for a tool, or what that tool returned — it would ask again, forever.
- **When:** every ReAct loop, always. This is not an optional or advanced feature — it is the loop's only memory, and there is no working version of the loop without it.

Two real ways to structure a growing scratchpad, and when each earns its cost:

- **Full raw history — keep every message. What:** append every request and result, verbatim, and never remove or shorten anything; re-send the whole list every turn. **Why:** the simplest possible correct behavior — nothing is ever lost, and the `tool_call_id` pairing below never

has anything torn out from under it. **When:** short runs, or runs whose tool results stay small — most of this document's own exercises, and most of Project 2's Worker runs, never come close to a real context problem.

- **A summarized/compressed history. What:** once the scratchpad gets long, replace older, already-acted-on turns with a short summary (model-written or rule-written), keeping the system prompt, the original task, and the most recent turns raw. **Why:** keeps a long run inside the context window, and keeps the model's attention on what's still relevant instead of buried under dozens of old, already-used tool results — the same "lost in the middle" risk Doc08's RAG topic names for retrieved text. **When:** long-running agents — many steps, or a Doc09-checkpointed run spanning minutes or hours — where a raw history would either blow the context window or bury the one fact a later step needs under noise nobody re-reads.

```
# Approach 1 — full raw history: simplest, correct, enough for most short runs
messages.append(response)
messages.append(
    {"role": "tool", "content": observation, "tool_call_id": call["id"]}
)
# nothing removed, ever — fine as long as the run stays short
# and results stay small

# Approach 2 — summarize older, already-acted-on turns once the scratchpad grows
def compress_if_needed(messages, keep_raw=6):
    system, task = messages[0], messages[1]
    # NEVER summarize these — still in play
    recent = messages[-keep_raw:]
    older = messages[2:-keep_raw]
    if not older:
        return messages
    prompt = (
        "Summarize what happened in these agent steps, in 3-5 sentences, "
        "keeping any concrete facts, numbers, or IDs found:\n"
        + format_for_summary(older)
    )
    summary = summarize_model.invoke(prompt).content
    earlier = {"role": "system", "content": f"Earlier steps so far: {summary}"}
    return [system, task, earlier, *recent]
```

Approach 1 costs nothing extra to write and never loses a detail — but it eventually hits the context wall below on a long enough run. Approach 2 buys headroom, at the real cost of one more model call per compression pass and a small risk the summary drops a detail a later step still needed — which is exactly why the most recent turns stay raw and untouched, never summarized.

- `tool_call_id` is the glue: it is how a `tool` message is understood as *the answer to* a specific earlier request, not a floating fact. Break that pairing — an orphaned `tool` message, or a

request with no matching reply — and the model loses the thread, or the API rejects the call outright.

- The context window is a hard wall you hit from **inside**: nothing is removed by itself, so a run that worked fine at step 5 can die at step 12 with a context-length error.
- Models attend most to the start and end of a long prompt. A fact buried in the middle of a very long scratchpad can be *present but ignored* — trimming is not only about cost.
- Tool results are untrusted text the moment a tool touches an outside source — a web page, a document, an email. That text enters the model's prompt through the scratchpad exactly like Doc06's prompt-injection topic describes; label tool results as data, never as instructions.
- Trimming and summarizing are the same family of fix, one cheaper and one smarter: trim before the model call, not while appending — keep system + first user message, drop older turns in **pairs** from the oldest end (never leave an orphaned `tool` message), cap each tool result's length. Summarize instead of trimming once even the dropped turns hold a fact a later step actually needs. Either way, log the full, untrimmed version — a cheap prompt and a complete record are not the same file.
- Scratchpads leak: they hold whatever the customer typed and whatever your tools returned, often names and IDs. If you log the scratchpad for debugging — and you should — redact first, and keep a short retention time.
- In a multi-agent system, "who owns the scratchpad?" is a real design choice — isolated scratchpads per agent (the usual default), one shared list every agent sees, or shared structured state plus private scratchpads. Doc11 is where this gets decided.
- **Applied to Project 2:** the Worker's `messages` list is this scratchpad, full stop. The Verifier (Step 4) deliberately never sees it — only the original question and the Worker's final answer — exactly so it judges the actual answer, not how hard the Worker "tried." That's the single-agent seed of Doc11's own "share the finding, not the scratchpad" rule.

Situation	What to do	Why
Short run, small tool results	Keep everything, raw	Simplest and safest; nothing to lose
A tool returns a long document	Trim or summarize before appending	The full text is re-sent every later turn
A tool failed	Append the error text as the result	The model needs to see the failure to fix its next move
Run is long, nearing the context limit	Keep system + task + last N turns; drop in pairs	An orphaned `tool` message breaks the `tool_call_id` pairing
A value must survive exactly (an ID, a total)	Typed state next to the messages	Prose is easy to misread; Doc09's state is exact

Common mistakes:

- *Mistake:* building a fresh `messages` list inside the loop instead of appending to the same one. → *Symptom:* the agent calls the same tool again and again with the same arguments until the step limit fires. → *Fix:* create the list once, outside the loop, and only ever append.
- *Mistake:* catching a tool error and appending nothing, or `"ok"`. → *Symptom:* the agent reports a confident, completely made-up success. → *Fix:* append the real error text — Doc06's rule again, now inside a loop where it matters on every turn.

Where you'll meet it: Doc03's no-memory topic is the same fact one level up; Doc06's prompt-injection topic is what a poisoned scratchpad entry becomes. The Build Task requires logging this exact list. Doc08 decides what retrieved text is worth putting into it; Doc09 turns it into graph state with a checkpointer; Doc11 makes shared-vs-private a system-design choice; Doc12 worries about redacting it before it reaches your logs; Doc13 turns each message into a trace span.

Quick cheat sheet:

- The scratchpad is one run's `messages` list. No list, no memory — Doc03's fact, still true here.
- `tool_call_id` pairs a result with the request it answers — never break the pair.
- Trim before the model call, log the full version — cheap prompt, complete record.
- Tool results are untrusted text entering your prompt: label them as data, not instructions.
- One scratchpad per agent by default; a multi-agent handoff passes a summary, not the whole log.

Setting a limit on the number of steps

The loop needs a way to know it's done: the model gives a final answer with no more tool calls, or an outside limit is hit. This is the same "a limit isn't optional" discipline as Doc02's retry topic — a fixed attempt cap, then raise, never `while True` trusting it will work out. A step limit is your agent's circuit breaker: the wiring is supposed to be fine, the model is supposed to notice when it has enough and stop — the limit exists for the day it doesn't, tripping not because anyone predicted *this* fault but because someone decided in advance that past a point, stopping beats continuing.

How it really works

What/Why/When, named directly:

- **What:** a hard, code-enforced ceiling on how much one agent run is allowed to spend — checked in your own code, never left to the model's own judgment about when it's done.
- **Why:** the model is *supposed* to notice it has a final answer and stop. "Supposed to" is not a guarantee — the limit exists for the day it doesn't, the same reasoning as Doc02's retry cap: a plan that only works when nothing goes wrong is not a plan.
- **When:** every agent loop, no exceptions. There is no safe agent loop without one, the same way there is no safe retry loop without a max-attempts cap.

Three real ways to measure "has this run had enough," and why you layer all three instead of picking one:

- **A simple step counter. What:** `for step in range(max_iterations):` — count model calls, stop cold at the number. **Why:** free, instant, exactly predictable — you always know the worst case, and it needs no extra measurement, just the loop's own counter. **When:** the always-on default layer, for every agent loop, including this document's Build Task — a fast loop making cheap calls hits this one first.
- **A wall-clock (time) budget. What:** a `time.time()` deadline checked at the top of each loop iteration, stopping once real elapsed time crosses a limit — regardless of how many steps that took. **Why:** a step counter alone never catches one slow tool call — 6 steps can still take 10 minutes if a single call hangs, and the same "never one call without a timeout" discipline Doc02's timeouts topic teaches for one HTTP call applies to a whole run. **When:** any agent behind a live user request (nobody waits happily past a few seconds) or any background/scheduled job (it must finish, or fail, inside its window).
- **A token/cost budget. What:** add up `response.usage_metadata["total_tokens"]` (or a $ estimate from it) across the run, and stop once a set budget is spent. **Why:** a scratchpad that grows every step (the previous topic) means every later step costs more than the one before it, at a *fixed* step count — Doc03's cost topic is exactly why cost can blow past a sane number long before the step counter does, especially once tool results get large. **When:** any paid API in front of real traffic, or any agent whose tool results vary a lot in size (search results, whole documents).

```
# Approach 1 — a simple step counter: free, instant, the always-on base layer
def run_agent(task: str, max_iterations: int = 6) -> str:
    messages = [{"role": "user", "content": task}]
    steps = []
    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        # ...same THINK/ACT/OBSERVE body as the ReAct loop's code above...
    message = f"no final answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

```
# Approach 2 — layer a wall-clock deadline under the step counter
import time


def run_agent(task: str, max_iterations: int = 6, max_seconds: int = 30) -> str:
    deadline = time.time() + max_seconds
    messages = [{"role": "user", "content": task}]
    steps = []
    for step in range(max_iterations):
        if time.time() > deadline:
            message = f"time budget ({max_seconds}s) used up at step {step}"
```

```
            raise MaxIterationsExceeded(message, steps)
        response = model_with_tools.invoke(messages)
        # ...same THINK/ACT/OBSERVE body...
    message = f"no final answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

```
# Approach 3 — add a token budget, the layer a growing scratchpad needs
def run_agent(
    task: str,
    max_iterations: int = 6,
    max_seconds: int = 30,
    max_tokens: int = 20_000,
) -> str:
    deadline = time.time() + max_seconds
    total_tokens = 0
    messages = [{"role": "user", "content": task}]
    steps = []
    for step in range(max_iterations):
        if time.time() > deadline:
            message = f"time budget used up at step {step}"
            raise MaxIterationsExceeded(message, steps)
        if total_tokens > max_tokens:
            message = f"token budget ({max_tokens}) used up at step {step}"
            raise MaxIterationsExceeded(message, steps)
        response = model_with_tools.invoke(messages)
        # how: usage_metadata is LangChain's name for Doc04's usage
        tokens = response.usage_metadata["total_tokens"]
        total_tokens = total_tokens + tokens
        # ...same THINK/ACT/OBSERVE body...
    message = f"no final answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

All three raise the same `MaxIterationsExceeded`, carrying the partial log — the *shape* of hitting a budget never changes, only which measurement tripped it. A step counter costs nothing and catches a stuck loop; a wall-clock deadline costs one `time.time()` check and catches a hung call the counter misses entirely; a token budget costs reading `.usage_metadata` off every response and catches an expensive scratchpad the other two never notice. None of the three replaces the other two — that's why production code layers all three, not the "pick the best one" instinct a first pass usually reaches for.

- The loop ends only two ways: **natural** — the model replies with text and no `tool_calls` — or **forced** — one of the budgets above runs out. Relying only on the natural end bets production on a system that also hallucinates.
- The counter wraps the **model call**, not the tool call. One reply can carry zero tool calls or several, so counting tool calls does not match the number of paid round trips you are actually

making.

- When the limit fires you have three honest choices: raise a **named error carrying the partial log** (best default — loud and debuggable), return a **partial result marked incomplete** (user-facing chat), or spend **one last call with `tool_choice="none"`** to force a real answer from whatever was already gathered. Catching the error and returning a generic "something went wrong" with nothing logged is not a fourth choice — it turns a fixable bug into an invisible one.
- `recursion_limit` in `create_agent` counts graph steps, roughly **two per ReAct turn** — `recursion_limit=10` is about 5 tool rounds, not 10. The graph mechanics behind this number belong to Doc09.
- Duplicate-call detection — hash `(tool_name, arguments)` — catches the "asking for the exact same thing again" stutter, often before the step cap even fires.
- In a multi-agent system, budgets must be **global**, not per agent. A 5-agent system with a 10-step cap each has an effective ceiling of 50 model calls, and if one agent can call another as a tool, that ceiling multiplies instead of adding. Doc11 is where the shared budget lives.
- "Limit hit" is a metric to track, not just an exception. A healthy agent should trip it rarely (well under 1% of runs); a high rate means a tool description or system prompt needs fixing, not that the number should go up.
- **Applied to Project 2's Worker:** the Build Task's requirement is Approach 1 at minimum — a step counter, always. Add Approach 2 (a wall-clock deadline) the moment a real user is waiting on the Worker, and Approach 3 (a token budget) once the tool set includes anything that can return a large, unpredictable amount of text.

Option	What it does	Mandatory?
`max_iterations` (your counter)	Caps model calls per run	Yes — no safe default exists
Wall-clock deadline	Caps total seconds	Yes for any background or scheduled job
Token / cost budget	Caps tokens spent per run	Yes for paid APIs, any user-facing loop
Duplicate-call detection	Stops a stuck repeat before the cap does	Strongly recommended
Per-tool call cap	Caps one dangerous action (e.g. `send_refund`) per run	Yes for any write tool

Common mistakes:

- *Mistake:* catching the limit error and returning a friendly message with nothing logged. → *Symptom:* users report "the bot gives up sometimes," and the logs show nothing at all. → *Fix:* log the full step log at `ERROR` (Doc01's logger), *then* show the friendly message.
- *Mistake:* shipping only a step counter and calling that "the" step limit. → *Symptom:* one slow tool call turns a 3-step run into a 4-minute run, with the step counter never close to firing. → *Fix:* layer

a wall-clock deadline (Approach 2 above) under the step counter — they catch two different failures, not the same one twice.

- *Mistake:* assuming `recursion_limit=10` in `create_agent` means 10 tool rounds. → *Symptom:* `GraphRecursionError` after only about five tool calls. → *Fix:* one ReAct turn costs roughly two graph steps — set `recursion_limit` to about `2 × desired_rounds + 2`.

Where you'll meet it: Doc02's retry-limit discipline and timeout discipline are the same two rules, one layer down. Doc03's cost topic is exactly why a token budget earns its own layer instead of just a bigger step count. The Build Task requires a `MaxIterationsExceeded` error with the partial log attached, and the Failure exercise makes you watch a loop run before restoring the limit. Doc09 has the same guard as `recursion_limit`; Doc11 turns it into a shared budget across agents; Doc12 pairs it with request timeouts and rate limits; Doc13 makes "share of runs that hit the limit" a dashboard number.

Quick cheat sheet:

- Every loop gets a hard limit in your code. The model's judgment is not a stopping condition — same rule as Doc02's retry cap.
- Count model calls, not tool calls.
- Raise a **named** error and attach the partial step log — "it hung" is not a bug report.
- Layer three budgets: a step counter (free, always on), a wall-clock deadline (catches a hung call), a token/cost budget (catches an expensive scratchpad — Doc03's cost fact is why).
- `recursion_limit` counts graph steps: roughly two per ReAct turn.
- A high "limit hit" rate is a prompt/tool bug, not a reason to raise the limit.

Common single-agent failures, worth naming now

Once a loop runs many steps, three failures show up again and again, and naming them now makes them recognizable later instead of feeling like new mysteries each time. **Wrong tool arguments**: Doc06's Pydantic check confirms the *shape* the model sent, not that the values are *correct* — a validated but wrong city still validates. **Ignoring a failure**: a tool returns an error, but the model carries on as if it worked and states a made-up result as fact — exactly why Doc06's rule to return clear, sentence-shaped errors matters, though it only gives the model a real chance to react correctly, not a guarantee. **Infinite loop**: covered in full in the topic above.

How it really works

Each failure, named directly — what it is, why it happens, when to expect it:

- **Wrong tool arguments. What:** the model calls the right tool, with arguments that pass Pydantic's shape check but are still factually wrong — `get_weather(city="Lahroe")`, a real-looking typo, or an order ID that's one digit off. **Why:** Doc06's Pydantic contract only proves the *shape* is right — a string where a string belongs — it has no way to know "Lahroe" isn't a real

city. **When:** any tool whose arguments come from parsing free-text input rather than a fixed, closed set of choices — the vaguer the input, the more this shows up.

- **Ignoring a tool failure. What:** a tool call fails, the model receives the error text, and it carries on anyway, stating a made-up result as if the call had worked. **Why:** if the error string reads like a plausible result (`""`, `"ok"`, or nothing at all), the model has nothing that clearly says "this failed" — Doc06's rule to return a clear, sentence-shaped error exists specifically to give it that signal, though even a clear error is a chance to react correctly, not a guarantee. **When:** any tool that can genuinely fail at runtime (a flaky API, a timeout, a bad lookup).

- **The infinite loop. What:** the model keeps calling tools — often the identical call — with no final answer ever arriving. **Why:** nothing in the model's own reasoning forces it to notice "I'm not making progress"; that recognition is exactly the job Setting a limit on the number of steps does from the outside. **When:** an ambiguous task with no clean stopping signal, or a tool result that doesn't obviously answer the question it was asked.

- Each of these three has a distinct symptom, which is the fastest way to tell them apart from a log: a confidently wrong answer usually means bad arguments; a confidently *invented* success usually means a swallowed tool error; a spinning step count or a hit step limit means the loop.

- Wrong arguments pass Pydantic because shape and correctness are different guarantees — Doc06's "Checking arguments" topic states this directly: shape ≠ safety, and here, shape ≠ correctness either.
- An ignored failure is a scratchpad problem as much as a tool problem — if the tool's error string reads like a real answer (`""`, `"ok"`), the model has nothing to tell the two apart with.
- Logging every step — tool name, arguments, result, step number — is what turns "it doesn't work" into "step 3 called `refund` with the wrong order ID." The Build Task's step log exists specifically for this.
- These three are exactly what Doc14's debugging lab will have you diagnose from symptoms alone, once you have a working agent to break on purpose.
- **Applied to Project 2:** Step 3 deliberately requires a tool "set up to sometimes fail on purpose" — that requirement exists specifically to force you to watch failure #2 happen live and confirm your recovery path actually works, not just reads correctly on paper.

Failure	Typical symptom	Where the fix lives
Wrong tool arguments	Confident, plausible, wrong answer	Doc06's Pydantic contract catches shape; you must sanity-check values
Ignored tool failure	Confident, invented success	Doc06's "return a clear error string" rule
Infinite loop	Step count maxes out, or it visibly stutters	This document's step-limit topic

Common mistakes:

- *Mistake:* assuming a Pydantic-valid tool call is a *correct* one. → *Symptom:* a wrong but perfectly plausible-looking answer. → *Fix:* validation is shape only — sanity-check the returned value itself, not just that the call went through.
- *Mistake:* not logging enough to tell these three apart. → *Symptom:* "the agent is buggy sometimes" reports with no way to diagnose which failure it was. → *Fix:* log tool name, arguments, result, and step number on every turn, always.

Where you'll meet it: Doc06's Pydantic contract and failure-handling topics are the fixes for the first two. The Build Task's step-log requirement is what makes all three diagnosable, and Doc14 is a whole document of exercises built on exactly these three.

Quick cheat sheet:

- Wrong arguments, ignored failures, and infinite loops are the three failures worth knowing by name.
- Each has its own symptom — use the symptom to guess the cause before you dig into logs.
- A step log (tool, args, result, step number) is what makes any of the three diagnosable at all.

Agent memory: what actually persists between calls

The scratchpad is short-term memory only — it lives for one run and disappears the instant that run ends. A real assistant often needs to remember something *across* separate conversations — a name, a past preference, a fact from last week — and the scratchpad genuinely cannot do that, because a brand-new run starts with a brand-new, empty `messages` list.

How it really works

What/Why/When, named directly:

- **What:** the real question here isn't "does the agent remember" — it's "where does that memory live, and does it survive the process ending." The scratchpad (previous topic) *is* real memory, just for one run only.
- **Why:** a brand-new run starts with a brand-new, empty `messages` list — Doc03's no-memory fact means nothing survives inside the model itself, and nothing survives inside a Python list once the process holding it ends.
- **When:** the moment a product needs to remember something *across* separate conversations — a returning user's name, a stated preference, a fact from last week — a scratchpad-only design cannot do this, no matter how long the scratchpad is allowed to grow.
- This is Doc03's "no memory inside the model" fact again: the model never remembers anything on its own, whether it's this run's scratchpad or a fact from last week — both are context your code has to re-inject.
- A longer scratchpad is not memory. It still dies with the process; real memory has to survive a restart, which means it lives somewhere outside your running program — a database, a file, a

vector store.

- Long-term memory means deciding **what** is worth saving (not a full transcript of everything, forever — that is neither useful nor readable later), **where** it is stored, and **how** it re-enters a *future* run's context without dumping the whole history back in and blowing past the context window.
- This document deliberately does not solve that problem — the real mechanism needs a graph to hang state on, and belongs to Doc09's "Long-term memory: remembering across separate conversations" topic.

	Scratchpad (this document)	Long-term memory (Doc09)
Lives for	One run	Across runs and conversations
Storage	A Python list in memory	A database, file, or vector store
Survives a restart	No	Yes

Common mistakes:

- *Mistake:* assuming a longer chat history is the same thing as memory. → *Symptom:* the assistant still "forgets" everything the moment the process restarts. → *Fix:* memory that must survive a restart lives outside the running program — that's a storage decision, not a bigger scratchpad.
- *Mistake:* planning to save every message "just in case." → *Symptom:* an unreadable dump nobody can use to decide what a future run actually needs. → *Fix:* decide what's worth saving on purpose — curated facts, not a full transcript.

Where you'll meet it: Doc03's no-memory topic is the same fact this topic extends. Doc09 is the real mechanism, once there's a graph to hang it on. Project 10 builds this exact system end to end.

Quick cheat sheet:

- The scratchpad is short-term only; it dies with the run.
- Long-term memory is a storage decision — what to save, where, and how it re-enters a future run's context.
- Not solved here on purpose — see Doc09 once you have a graph to attach it to.

Planning before acting

The ReAct loop decides one step at a time — think, act, observe, then decide the *next* step only once the real result is in. An alternative has the model write a multi-step plan up front, before doing anything, then execute that plan step by step.

How it really works

What/Why/When for each of the two shapes:

- **ReAct — decide as you go (this document's approach). What:** think, act, observe the real result, then decide only the next step. **Why:** naturally adapts the moment a result surprises you — nothing planned in advance ever needs to be thrown away. **When:** the default, whenever a task's steps genuinely can't be fully known before the first real result comes back.
- **Plan-first. What:** the model writes a full multi-step plan before running anything, then that plan gets executed by step. **Why:** a plan written down is a plan a human can read and approve *before* anything real happens — genuinely more reviewable than a loop that starts acting immediately. **When:** review-before-action matters more than adapting on the fly — often paired with Doc10's `interrupt()` for the actual human check.
- Deciding one step at a time reacts well to surprises — an unexpected tool result naturally changes what happens next — but it can wander, taking a longer, less predictable path.
- Planning first is more predictable and reviewable: a human can read the plan before any tool actually runs. It costs an extra model call up front, and a plan made before seeing any real result can turn out wrong the moment the first result arrives, needing a re-plan anyway.
- Plan-first does not remove the need for a step limit from the topic above — a bad re-plan loop can still repeat forever, just one layer up.
- A plan is not a guarantee it will be followed exactly: a later tool result can invalidate step 3 of a 5-step plan, and code that blindly executes the stale plan anyway produces a confidently wrong answer.
- Doc10 formalizes this as a named workflow pattern ("plan and execute") once you have more patterns to choose between.

	ReAct (this document)	Plan-first
When to decide the next step	After seeing the last real result	All steps decided up front
Adapts to a surprising result	Naturally	Needs a re-plan
Reviewable before anything runs	No	Yes
Extra up-front cost	None	One extra model call for the plan

Common mistakes:

- *Mistake:* assuming plan-first removes the need for a step limit. → *Symptom:* a bad re-plan loop runs just as long as an unguarded ReAct loop would have. → *Fix:* the step-limit discipline applies to both designs equally.
- *Mistake:* treating a written plan as something that will be followed exactly. → *Symptom:* code executes a stale step even after an earlier result made it wrong. → *Fix:* re-check the plan against real results before running each step, or re-plan when a result contradicts it.

Where you'll meet it: Doc10 contrasts plan-first with ReAct and fixed workflows as named patterns; a supervisor in Doc11 sometimes plans before delegating to specialists.

Quick cheat sheet:

- ReAct: decide as you go. Plan-first: decide up front, then execute.
- Plan-first is more reviewable, costs one extra call, and can still need a re-plan.
- Both designs need the same step-limit discipline — planning is not a safety mechanism.

When NOT to use an agent

An agent loop earns its complexity only when the number and order of steps genuinely can't be known in advance — when what happens next truly depends on what the last tool returned.

How it really works

Two real ways to decide this for a whole task, before any loop starts — and when to pick each:

- **A fixed rule, decided at design time. What:** you, the developer, look at a task type and decide in advance — "invoice reconciliation is always fetch → compare → format, write it as a function" — no runtime check at all; the decision is baked into which code path a request even reaches. **Why:** free, instant, and fully testable — there's no routing decision that can itself be wrong, because there's no routing. **When:** a system that only ever serves one or a few known task types, where real examples today show the step order never varies — the common case, and the default.
- **A routing model, decided per request at runtime. What:** a small, cheap model call (structured output, the same shape as Doc10's routing topic) classifies an incoming request as `"fixed_pipeline"` or `"needs_agent"` before either path runs. **Why:** a system serving a genuine mix of request types can't hard-code the split in advance, because the split depends on what each request actually asks for. **When:** a real product surface where you can't fully enumerate task types ahead of time, and misrouting a fixed-shape task into an expensive loop (or the reverse) shows up often enough in your logs to be worth a routing call.

```
# Approach 1 — a fixed rule, decided once, at design time (no runtime check)
def handle_request(task: str) -> str:
    # invoice reconciliation is ALWAYS fetch -> compare -> format; decided by
    # looking at real past tasks, not guessed
    return run_invoice_pipeline(task)   # a plain function / Doc05's LCEL chain
```

```
# Approach 2 — a small routing model decides per request, at runtime
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class ShapeDecision(BaseModel):
    needs_agent: bool
    reason: str
```

```
router_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0,
).with_structured_output(ShapeDecision)

def handle_request(task: str) -> str:
    prompt = (
        "Can this task's steps be written down today, in a fixed order, "
        f"with confidence they won't change along the way? Task: {task}"
    )
    decision = router_model.invoke(prompt)
    if decision.needs_agent:
        return run_agent(task, max_iterations=6)       # this loop
    return run_fixed_pipeline(task)                            # Doc05's chain
```

Approach 1 costs nothing and can never misroute, but only works once real traffic has actually confirmed the task type never varies. Approach 2 costs one small model call per request and can itself be wrong — the same "silent wrong decision, not a crash" risk Doc10's own router carries — but it's the only real option once traffic is a genuine, unpredictable mix.

- If a task always takes the same fixed sequence — always tool A, then always tool B, then format the result — that's not a job for a loop deciding each step on its own. Write it as a plain function or a fixed pipeline, the same "no loop, no model in charge" case as Doc05's LCEL chains.
- An agent loop costs more (each step is its own paid model call), is slower, and is less predictable than code that runs the same steps in the same order every time — and none of that cost buys anything if the order was never actually in question.
- One tool, one call, then answer needs no loop at all — that's just Doc06's single round.
- The simple test: if you can write the sequence of steps down today, confident it won't change based on what happens along the way, write that sequence as code. Reach for a loop only once you genuinely can't.
- Wrapping an already-deterministic pipeline in an agent "to be flexible" is a real anti-pattern — it buys nondeterminism and cost for a task that never varies.
- **Applied to Project 2's Worker:** each individual tool the Worker calls (a lookup, a calculator) could be one Doc06 single call on its own — the Worker as a whole still earns the loop, because *which* tool runs next, and how many times, genuinely isn't fixed in advance for the questions Project 2 is built to answer.

Situation	Use	Why
Steps always the same, same order	Plain function or fixed pipeline (Doc05)	Cheaper, faster, fully testable
Next step depends on the last result	Agent loop (this document)	Only the model can decide after seeing the real result

Situation	Use	Why
One tool, one call	Doc06's single round	A loop that always turns once is just overhead
Steps knowable, but must be reviewed first	Plan-first	A human approves before anything runs

Common mistakes:

- *Mistake:* giving every task an agent loop "to be safe." → *Symptom:* a task that never varies costs 3-8× more and runs slower for zero benefit. → *Fix:* if the order is always the same, write it as code.
- *Mistake:* reaching for an agent because it sounds more advanced. → *Symptom:* a deterministic job becomes harder to test and its output stops being repeatable. → *Fix:* the simple test — can you write the steps down today with confidence they won't change.
- *Mistake:* using an LLM router (Approach 2) for a decision that's really just a fixed rule dressed up as a judgment call. → *Symptom:* paying for a classification call on every request, for a routing decision that was always going to land the same way. → *Fix:* try Approach 1 first — only reach for Approach 2 once real traffic actually shows a task-type mix a fixed rule can't capture.

Where you'll meet it: Doc05's LCEL chains are the fixed-pipeline alternative; Doc06's single round is the one-call alternative. Doc10's routing topic is this exact rules-vs-model-call choice, one level down, deciding "should I search" instead of "does this whole task need a loop." Doc10 as a whole is an entire document about choosing the right shape for a task; Doc11 asks the same question again, one level up, about whether a task needs *several* agents.

Quick cheat sheet:

- An agent loop earns its cost only when the step order genuinely can't be known in advance.
- Fixed order → plain code (Doc05's chains). One call → Doc06's single round. Unknown order → this document's loop.
- Decide the shape with a fixed rule (free, the default) or a small routing model (for a genuine, unpredictable mix) — the same choice Doc10's own search-router makes one level down.
- "Sounds more advanced" is not a reason to add a loop.

Evaluating an agent, not just a single call

Checking whether an agent's *final answer* is correct isn't enough to trust it in production — an agent can land on the right answer by an expensive, fragile, or lucky path, and that path is exactly what breaks next time the input changes slightly.

How it really works

Two real ways to check a run beyond its final answer, and when to pick each:

- **A rule-based checker. What:** plain code checks — did the run call the *expected* tool for this fixture prompt? Was the step count inside a normal range? Did any `(tool, arguments)` pair repeat? **Why:** free, instant, fully deterministic — the same run always gets the same verdict, nothing to disagree about. **When:** you can write down, in advance, what "right" looks like for a fixture prompt — the common case for a stable, well-understood task.
- **An LLM-judge. What:** a separate model call is shown the task, the step log, and the final answer, and asked a structured question — did this take a reasonable path to a correct answer, yes or no, and why? **Why:** some quality judgments genuinely resist a fixed rule — "was this a *sensible* path" is a judgment call, not a checklist, the same reason Doc11's Reviewer agent exists at all. **When:** the thing you're checking is closer to judgment than fact — reasonableness, tone, whether an explanation actually makes sense — not something a rule can check directly.

```
# Approach 1 — a rule-based checker: deterministic, no extra model call
def check_run(log, expected_tool, max_reasonable_steps):
    tools_used = []
    for entry in log:
        tools_used.append(entry["tool"])
    seen, repeats = set(), 0
    for entry in log:
        key = (entry["tool"], str(entry["arguments"]))
        if key in seen:
            repeats += 1
        seen.add(key)
    return {
        "used_expected_tool": expected_tool in tools_used,
        "step_count_ok": len(log) <= max_reasonable_steps,
        "repeats": repeats,
    }
```

```
# Approach 2 — an LLM-judge, for judgment calls a rule can't express
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class RunJudgment(BaseModel):
    took_reasonable_path: bool
    reason: str

judge_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0,
).with_structured_output(RunJudgment)

def judge_run(task: str, log: list[dict], final_answer: str) -> RunJudgment:
    return judge_model.invoke(
        f"Task: {task}\nSteps taken: {log}\nFinal answer: {final_answer}\n"
        "Was this a reasonable, non-wasteful path to a correct answer? "
```

```
        "Explain briefly."
    )
```

Rule-based catches exactly what you thought to check, for free, every time. An LLM-judge catches the vaguer "this technically worked but the path was bad" case a rule can't state — at the cost of one more model call per evaluation, and a judge that can itself be inconsistent unless pinned at `temperature=0`. Most real eval suites run both: rules for the facts (tool choice, step count), a judge for the judgment call (was this actually a good answer).

- A wrong intermediate step that happens to get corrected later still cost real time and money, and a run like that is more likely to fail outright on the next similar task.
- Check beyond the final answer: did it use the **right tools**, not a wrong one that happened to still work; did it take a **reasonable number of steps**, not an inflated one; did it avoid **looping** — the same or a very similar action repeated with no progress.
- Tool-use accuracy (right tool, right arguments) is the same measurement idea as Doc06's "selection accuracy is a metric, not a vibe" fixture — a small set of prompts with the expected tool for each, re-run on every change.
- Step-count sanity is simple but effective: a 2-step task that took 15 steps is a red flag even when the final answer happened to be right.
- This is exactly what Doc13's eval suite scores in production, turning "feels flaky" into a measured number.
- **Applied to Project 2:** the requirement to "confirm the Verifier actually catches at least one bad Worker answer" is Approach 2 in miniature — the Verifier agent in Step 4 *is* an LLM-judge, just judging one Worker run instead of a whole eval suite judging many.

What to check	How	Red flag
Tool choice	Compare against an expected-tool fixture	Right answer, wrong or extra tool used
Step count	Count model calls per run	Far more steps than similar successful runs
Repeats	Hash `(tool, arguments)` per run	Same call more than once with no new information
Final answer	Normal correctness check	Still necessary, just not sufficient alone

Common mistakes:

- *Mistake:* grading only the final answer. → *Symptom:* a fragile, expensive path looks fine until the next input trips it. → *Fix:* log and check tool choice and step count, not just the answer, the same log this document's Build Task already requires.
- *Mistake:* no baseline to compare against. → *Symptom:* "it got slower" or "it got worse" with nothing to point to. → *Fix:* keep a small fixture of prompts with expected tool/step-count ranges

— Doc06's selection-accuracy idea, applied to a whole run.

- *Mistake:* running an LLM-judge at a nonzero temperature and treating its verdict as ground truth. → *Symptom:* the identical run gets graded "good" one day and "bad" the next, with nothing about the run having changed. → *Fix:* `temperature=0` for any judge model, and treat its verdict as one more signal, not an infallible score.

Where you'll meet it: Doc06's tool-selection accuracy fixture is the same idea one level down. The Build Task's step log is the raw data these checks run over. Project 2's Verifier (Step 4) is Approach 2 built for real, one run at a time; Doc11's Generator → Critic → Revision pattern is the same LLM-judge idea, now inside the loop instead of after it. Doc13 is the full document on evaluation and observability; Doc14 has you diagnose a bad run from its logs alone.

Quick cheat sheet:

- A correct final answer is not proof of a good run — check the path too.
- Check tool choice, step count, and repeats, not just the answer.
- Rule-based checks for facts (free, deterministic); an LLM-judge for judgment calls a rule can't express (costs a call, pin `temperature=0`).
- Keep a small fixture with expected tools/step counts — the same discipline as Doc06's selection accuracy, one level up.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- ReAct: Synergizing Reasoning and Acting in Language Models (arXiv) — the paper behind the think→act→observe loop.
- LangChain — Agents concept — `create_agent`, the current standard way to get a ready-made tool-calling agent. Its older predecessor, `AgentExecutor`, is now legacy, but still worth recognizing if you read older code.
- Anthropic — Building Effective Agents — read the "agents" section again now that you know tool-calling.

Practice Exercises

Setup: same venv as before — if it's not active, `cd 07_ai_agents && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New packages for this document: `pip install langchain langchain-openai langgraph pytest`. Copy Doc06's Build Task files into `practice/`, unchanged: `tools.py`, `http_client.py`, `exceptions.py`, `logging_setup.py` — every exercise here drives those same tools.

Where your code lives: all of it under `07_ai_agents/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — the same convention as Doc01/Doc02/Doc06 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── react_loop_practice.py        Basic + Intermediate: run_agent()
├── react_loop_raw_sdk.py         Intermediate, Approach 5: raw SDK
├── agent_executor_comparison_practice.py   Real-world
├── no_tool_needed_practice.py    Edge cases
├── test_no_tool_needed.py        Edge cases, Approach 2: pytest
├── loop_safety_cost_practice.py  Failure
├── tools.py                      copied from Doc06, unchanged
├── http_client.py                copied from Doc06, unchanged
├── exceptions.py                 copied from Doc06, unchanged
├── logging_setup.py              copied from Doc06, unchanged
└── build_task/                   Build Task — its own folder
    ├── agent.py                  run_agent(), AgentResult, AGENT_TOOLS
    ├── docs_search.py            search_docs — fails once, on purpose
    ├── main.py                   reads a task, prints answer + steps
    ├── test_agent.py             the 4 Test Cases (plain, or pytest)
    ├── exceptions.py             Doc06's + MaxIterationsExceeded
    └── tools.py, http_client.py, logging_setup.py   copied from Doc06
```

Why each script exists:

- `react_loop_practice.py` — the loop itself, traced by hand first, then built for real — the one file every other exercise and the Build Task assumes you already have working.
- `agent_executor_comparison_practice.py` — proves `create_agent` is your same loop, not a different idea, by running both on the identical prompts.
- `no_tool_needed_practice.py` — confirms the loop doesn't waste a tool call when it doesn't need one.
- `loop_safety_cost_practice.py` — the only place you watch a loop actually run away, and measure what that costs, before the step limit goes back on.
- `react_loop_raw_sdk.py` — the same loop without LangChain, so you see what `bind_tools()` writes for you.
- `test_no_tool_needed.py` — the no-tool check as a pytest test — your first pytest file.
- `tools.py`, `http_client.py`, `exceptions.py`, `logging_setup.py` — Doc06's tool library, copied unchanged, so every exercise drives the same tools.

For this document, save your practice code as:

- **Basic** (trace the loop on paper first) and **Intermediate** (build the loop yourself) are both about the ReAct loop itself — save them together as `practice/react_loop_practice.py`, one section per level.
- **Real-world** (compare your loop to the library's) is its own topic — save it as `practice/agent_executor_comparison_practice.py`.
- **Edge cases** (a task that needs no tool at all) is its own topic — save it as `practice/no_tool_needed_practice.py`.
- **Failure** (watch it loop, then measure the cost of looping at all) is its own topic — save it as `practice/loop_safety_cost_practice.py`.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — trace the loop on paper first

- **What:** trace a 3-step ReAct loop by hand on paper (prompt → thought → action → result → thought → final answer) before writing any code.
- **Why:** if you can't trace it on paper, you can't debug it in code — the cheapest possible way to catch a wrong mental model.
- **Save as:** `practice/react_loop_practice.py`, under a `# Basic` section (this file also holds the Intermediate exercise, in its own section).
- **Builds on:** Doc06's round-trip topic — the same five steps, now traced by hand instead of run in code.
- **Used later by:** the Intermediate exercise right below, which turns this exact trace into real code.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — build the loop yourself

- **What:** build the ReAct loop yourself (no `create_agent`) for one tool.
- **Why:** this document's whole point is that the loop isn't magic — building it once, yourself, is what makes that true.
- **How to code it:** `model.bind_tools([...])`, then a `for step in range(max_iterations):` loop — ask the model; if it asked for tools, run each with `.invoke()` and append a `{"role": "tool", ...}` message (Doc06's round trip); if not, return its answer. Raise `MaxIterationsExceeded` with the step log if the loop runs out. Then: a factory that builds a fresh fail-once tool, your own Pydantic check before a tool runs (`model_dump()`), and the same loop on the raw OpenAI SDK.
- **Save as:** `practice/react_loop_practice.py`, under an `# Intermediate` section (this file also holds the Basic exercise, in its own section).
- **Builds on:** the Basic trace in this same file, and Doc06's Build Task tool library — register those same tools here.
- **Used later by:** the Build Task's `agent.py`, which is this exact loop made production-shaped with a step limit and tool-failure recovery.

- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — compare your loop to the library's

- **What:** switch to `create_agent` (the current standard library function for this) with your Doc06 tools registered, and compare its behavior to your own hand-built loop.
- **Why:** now that you've built the primitive, seeing the library's version side by side tells you exactly what it's doing for you, and what it's hiding.
- **How to code it:** wire the same tools into both your loop and `create_agent`, run the same 3 test prompts through each, and diff the final answers and step counts. Set `recursion_limit` and catch `GraphRecursionError` — remember it counts roughly two graph steps per ReAct turn. Then watch the library's steps live with `agent.stream(inputs, stream_mode="updates")`.
- **Save as:** `practice/agent_executor_comparison_practice.py`.
- **Builds on:** the Intermediate hand-built loop in this same folder — same tools, same prompts, a different engine underneath.
- **Used later by:** the Build Task, which expects you to understand both the hand-built loop and what `create_agent` does underneath it.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — a task that needs no tool at all

- **What:** a prompt with no correct tool to call — check whether the agent answers directly instead of forcing a tool call anyway.
- **Why:** an agent that always reaches for a tool, even when it doesn't need one, wastes cost and time on every single call.
- **How to code it:** ask something the model can answer from general knowledge alone, run it through your loop, and confirm zero tool calls happened. Then write the same check as a pytest test (`test_...` functions with `assert`).
- **Save as:** `practice/no_tool_needed_practice.py`.
- **Builds on:** the Intermediate loop — same loop, a prompt chosen so nothing matches.
- **Used later by:** the Build Task's Test Cases row "task needs no tool."
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — watch it loop, then measure the cost of looping at all

- **What:** temporarily raise the step limit (with a real wall-clock safety net in place, never fully unlimited) and watch a prompt loop, then restore the real limit. Then measure the token/cost difference between a 5-step agent run and one direct call.
- **Why:** you need to have watched an agent loop run away once for the step-limit requirement to feel like something you're protecting yourself from, not an abstract rule.

- **How to code it:** set `max_iterations=1000` with a hard wall-clock timeout as the safety net — the same "never one call without a cap" discipline as Doc02's timeouts. Run an adversarial prompt, watch it loop, kill it, restore the real limit. Then compare logged token usage for the looping run against a single direct call on an easy task. Also try a timeout from the outside (a thread with `concurrent.futures`), and print how the `messages` list grows each step.
- **Save as:** `practice/loop_safety_cost_practice.py`.
- **Builds on:** the layered-limits idea from Setting a limit on the number of steps — steps and a deadline together, never steps alone.
- **Used later by:** the Build Task's `MaxIterationsExceeded` requirement and its "task designed to loop forever" test case.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Project 2: Tool-Using Agent

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: a single agent with 3+ tools, real checking of arguments, and graceful handling of a tool failure mid-run.

Requirements:

- Reuses the tool library from 06_tools_function_calling.
- A hard limit on steps — must always stop, never loop forever, even on a tricky input.
- At least one tool must be allowed to actually fail during a real run, and the agent must recover (retry, fall back, or report clearly) instead of crashing or making things up.
- Logs every step of the loop (tool, arguments, result, step number) for later debugging.

Inputs: any text task typed into the terminal. **Outputs:** a final answer, plus a full step-by-step log of how it got there. **Constraints:** no unlimited loops, for any input. Every tool call's arguments must pass Pydantic checking before the tool actually runs.

Builds on: the Intermediate loop (Approach 2 — `find_tool()`, `run_tool()`, the logger), the Edge cases no-tool check, and the Failure step limit with its log. `exceptions.py` reuses Doc01's named-error pattern as `MaxIterationsExceeded`, carrying the partial step log exactly as the step-limit topic requires.

Suggested files:

```
07_ai_agents/practice/build_task/
├── main.py              reads a task, prints the answer and every step
├── agent.py             run_agent(), AgentResult, AGENT_TOOLS
├── docs_search.py       search_docs — fails on its first call, on purpose
├── test_agent.py        the 4 Test Cases below
```

```
├── exceptions.py        Doc06's file + MaxIterationsExceeded
├── tools.py             copied from Doc06, unchanged
├── http_client.py       copied from Doc06, unchanged
└── logging_setup.py     copied from Doc06, unchanged
```

- `main.py` — **What/Why:** the thin entry point that turns typed input into one `run_agent()` call and prints its result — or the partial log, if the limit was hit.
- `agent.py` — **What/Why:** the Intermediate exercise's loop (Approach 2), with two Doc06 tools and the fail-once search.
- `docs_search.py` — **What/Why:** a tool that fails once on command, so "recovers from a failure" is a repeatable test, not luck.
- `test_agent.py` — **What/Why:** proves the 4 Test Cases, in the plain `case` print style of Doc04–06 — run it every time `agent.py` changes.
- `exceptions.py` — **What/Why:** Doc06's named errors plus `MaxIterationsExceeded`, which carries the partial step log.
- `tools.py` / `http_client.py` / `logging_setup.py` — **What/Why:** Doc06's tool library, copied unchanged — the Pydantic `args_schema` on each tool is the argument check.

Run it: `cd practice/build_task && python test_agent.py` — from inside the folder, so the imports resolve. (With the pytest version: `pytest test_agent.py -v`.)

Functions/Components to build:

- `agent.py` → `run_agent(task: str, max_iterations: int) -> AgentResult`, with `find_tool()` and `run_tool()`
- `docs_search.py` → `search_docs(query)` and its `search_state` counter
- `exceptions.py` → `MaxIterationsExceeded(message, steps)`, raised (not silently ignored) when the limit is hit

Used later by: Doc09 rebuilds this same agent as an explicit graph; Doc10 wraps it in routing patterns; Doc11 runs several of these side by side.

Expected Behavior

- Clear tasks get solved in a few steps, using the right tool(s).
- A task with no relevant tool gets a direct answer, without forcing a tool call.
- A mid-run tool failure produces a clear, logged recovery — not a crash, not a made-up success.
- Hitting the step limit raises a clear, named error, with the partial log attached.

Test Cases

Scenario	Expected
Task clearly needs Tool A	Tool A is called, correct final answer
Task needs no tool	Direct answer, zero tool calls
A tool set to fail once then succeed	The agent recovers, reaches the correct final answer
A task designed to loop forever	The step limit is hit, `MaxIterationsExceeded` raised with the log

Break-It / Debug Preview

- Missing or wrong stopping condition → an endless loop.
- The agent ignores a tool's error and states a made-up result as fact.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- ReAct vs. plain function-calling · why step limits aren't optional in production · the cost of agent loops vs. single calls.

□ You Can Now Build Project 2

Docs 05-07 are everything Project 2 needs. Go to project_2_researchhand_tool_agent/ and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 2 survives all the break scenarios above, without you having to guide it by hand. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**